

Documentación

Antes de nada, confirma que Docker está instalado y accesible:

```
Desktop@DESKTOP-HTPKI6F MINGW64 ~  
$ docker --version  
Docker version 29.7.2, build a7dcaa6
```

Crear el proyecto

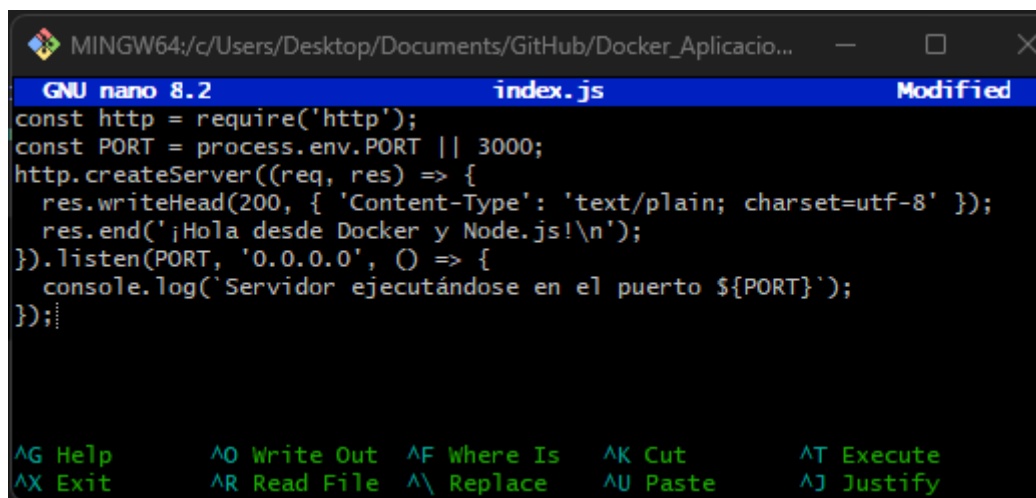
Primero se debe crear una nueva carpeta para la aplicación Node.js. Para ello, se utiliza la terminal y se ejecutan los comandos correspondientes para crear la carpeta y acceder a ella. Esta carpeta será la raíz del proyecto y contendrá todos los archivos necesarios para la aplicación.

```
Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js (main)  
$ mkdir hello-node  
  
Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js (main)  
$ cd hello-node
```

Crear la aplicación Node.js

Dentro de la carpeta del proyecto se crea un archivo llamado index.js. En este archivo se desarrolla un servidor HTTP sencillo utilizando Node.js. El servidor recibe las solicitudes y devuelve como respuesta el mensaje “¡Hola desde Docker y Node.js!”.

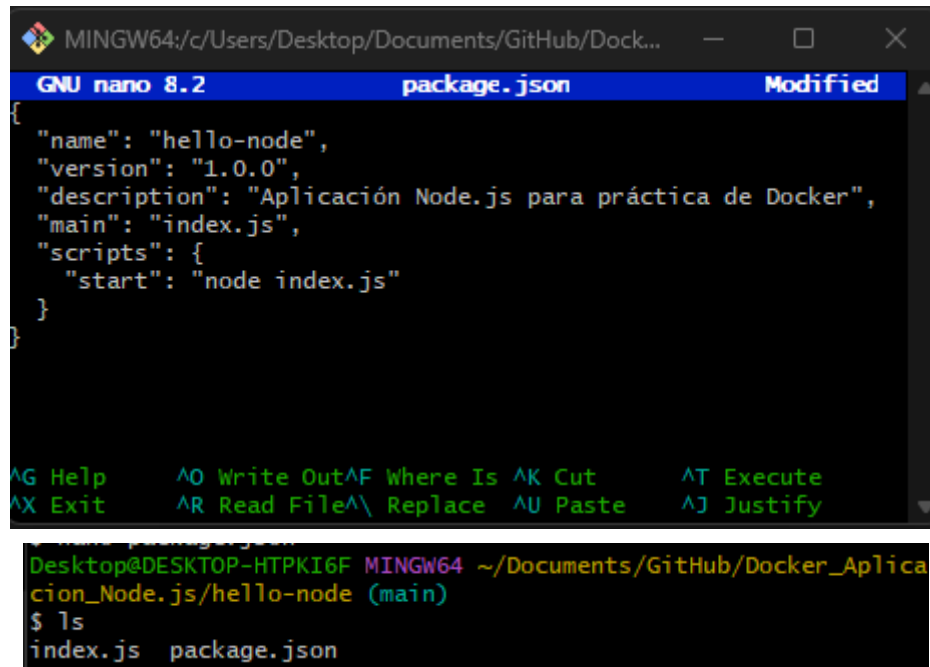
La aplicación utiliza el puerto 3000 de forma predeterminada, aunque también permite utilizar un puerto diferente mediante la variable de entorno PORT. Además, el servidor se configura para escuchar en 0.0.0.0, lo que permite que pueda ser accesible desde el exterior del contenedor Docker.



```
MINGW64:/c/Users/Desktop/Documents/GitHub/Docker_Aplicacio...  
GNU nano 8.2 index.js Modified  
const http = require('http');  
const PORT = process.env.PORT || 3000;  
http.createServer((req, res) => {  
  res.writeHead(200, { 'Content-Type': 'text/plain; charset=utf-8' });  
  res.end('¡Hola desde Docker y Node.js!\n');  
}).listen(PORT, '0.0.0.0', () => {  
  console.log(`Servidor ejecutándose en el puerto ${PORT}`);  
});  
  
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute  
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

Crear package.json

El archivo package.json contiene la información básica del proyecto Node.js, como su nombre, versión, descripción y archivo principal. También se define el script start, que permite iniciar la aplicación mediante el comando npm start.



```
MINGW64/c/Users/Desktop/Documents/GitHub/Dock...
GNU nano 8.2 package.json Modified
{
  "name": "hello-node",
  "version": "1.0.0",
  "description": "Aplicación Node.js para práctica de Docker",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  }
}

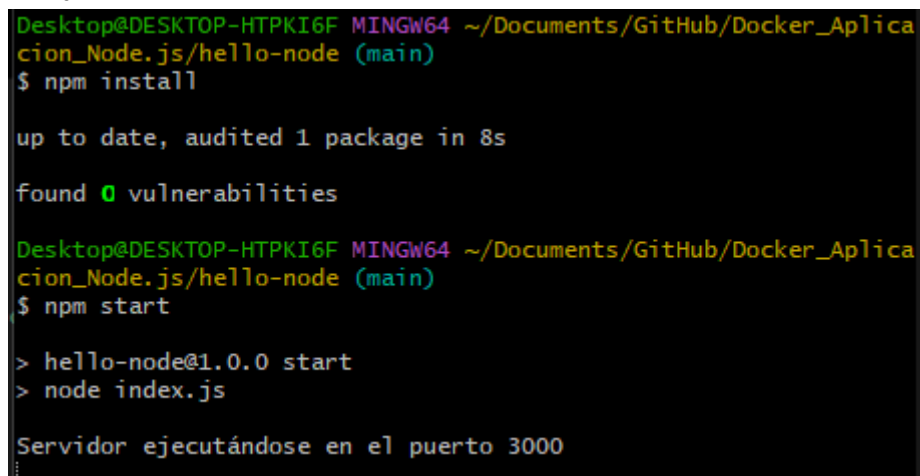
AG Help      AO Write Out AF Where Is AK Cut      AT Execute
AX Exit      AR Read File \ Replace AU Paste  AJ Justify

Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ ls
index.js package.json
```

Probar la aplicación localmente

Antes de utilizar Docker, se debe comprobar que la aplicación funciona correctamente de manera local. Para ello, se ejecuta npm install para instalar las dependencias y posteriormente npm start para iniciar el servidor.

Una vez iniciado el servidor, se puede acceder desde un navegador utilizando <http://localhost:3000>. Si todo funciona correctamente, se mostrará el mensaje “¡Hola desde Docker y Node.js!”.



```
Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ npm install

up to date, audited 1 package in 8s

found 0 vulnerabilities

Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ npm start

> hello-node@1.0.0 start
> node index.js

Servidor ejecutándose en el puerto 3000
```



¡Hola desde Docker y Node.js!

Crear el Dockerfile

Para dockerizar la aplicación se crea un archivo llamado Dockerfile en la raíz del proyecto. Este archivo contiene las instrucciones necesarias para construir la imagen Docker de la aplicación.

Se utiliza como imagen base node:20-alpine, se establece /app como directorio de trabajo, se copia inicialmente package.json y se ejecuta npm install. Después se copian los demás archivos del proyecto, se expone el puerto 3000 y se establece npm start como comando para iniciar la aplicación.

```
Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ cat > Dockerfile <<'EOF'
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
EOF

Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ ls
Dockerfile  index.js  package-lock.json  package.json
```

Construir la imagen Docker

Una vez creado el Dockerfile, se construye la imagen mediante el comando `docker build -t hello-node:1.0 ..`. Este comando crea una imagen llamada hello-node con la etiqueta 1.0, utilizando como contexto de construcción la carpeta actual.

Para comprobar que la imagen fue creada correctamente se utiliza el comando `docker images`.

```

Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ docker build -t hello-node:1.0 .
[+] Building 8.3s (11/11) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.1s
=> => transferring dockerfile: 148B                      0.0s
=> [internal] load metadata for docker.io/library/node:2 1.9s
=> [auth] library/node:pull token for registry-1.docker. 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                            0.0s
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:fb 4.9s
=> => resolve docker.io/library/node:20-alpine@sha256:fb 0.0s
=> => sha256:fff4e2c1b189bf87d63ad8bd07f7f4e 445B / 445B 0.2s
=> => sha256:b2cbbfe903b0821005780971ddc 1.26MB / 1.26MB 0.5s
=> => sha256:4feea04c154301db6f4a496ef 43.23MB / 43.23MB 3.7s
=> => sha256:6a0ac1617861a677b045b7ff885 3.86MB / 3.86MB 0.7s
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213 0.1s
=> => extracting sha256:4feea04c154301db6f4a496efa397b3d 1.0s
=> => extracting sha256:b2cbbfe903b0821005780971ddc5892e 0.0s
=> => extracting sha256:fff4e2c1b189bf87d63ad8bd07f7f4eb 0.0s
=> [internal] load build context                        0.1s
=> => transferring context: 968B                          0.0s
=> [2/5] WORKDIR /app                                 0.1s
=> [3/5] COPY package.json .                          0.0s
=> [4/5] RUN npm install                              0.6s
=> [5/5] COPY . .                                     0.0s
=> exporting to image                                 0.4s
=> => exporting layers                                    0.2s
=> => exporting manifest sha256:df42ebbd96c29e884af714db 0.0s
=> => exporting config sha256:da48830395017fa61f798972fa 0.0s
=> => exporting attestation manifest sha256:75c027b72dbf 0.0s
=> => exporting manifest list sha256:ce3fc24676804323a3f 0.0s
=> => naming to docker.io/library/hello-node:1.0          0.0s
=> => unpacking to docker.io/library/hello-node:1.0        0.1s

```

```

Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ docker images

```

IMAGE	ID	DISK USAGE	Info - U	In Use
hello-node:1.0	ce3fc2467680	193MB	48.4MB	

Ejecutar el contenedor

Después de construir la imagen, se ejecuta un contenedor utilizando el comando `docker run`. En este caso, el puerto 8080 del equipo local se conecta con el puerto 3000 del contenedor.

De esta manera, aunque la aplicación funciona internamente en el puerto 3000, se puede acceder a ella desde el equipo mediante `http://localhost:8080`.

```

Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ docker run -d \
  --name hello-node-container \
  -p 8080:3000 \
  hello-node:1.0
e4154d04f84a6525f658a762f80b0386f14007195632fbf39a730a270a4895cd

```



¡Hola desde Docker y Node.js!

Revisar el contenedor

Para comprobar que el contenedor se encuentra funcionando se utiliza el comando `docker ps`. Este comando permite visualizar los contenedores que están actualmente en ejecución. También se pueden consultar los mensajes generados por la aplicación mediante `docker logs hello-node-container` y obtener información detallada del contenedor utilizando `docker inspect hello-node-container`.

```
Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS        PORTS
NAMES
e4154d04f84a   hello-node:1.0 "docker-entrypoint.s..." 3 minutes ago
Up 3 minutes   0.0.0.0:8080->3000/tcp, [::]:8080->3000/tcp
hello-node-container
```

Detener y eliminar el contenedor

Cuando ya no sea necesario utilizar el contenedor, primero se puede detener mediante `docker stop hello-node-container`. Después, se elimina utilizando `docker rm hello-node-container`.

Finalmente, se puede utilizar `docker ps -a` para comprobar el estado de todos los contenedores, incluidos aquellos que ya fueron detenidos.

```
Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ docker stop hello-node-container
hello-node-container

Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ docker rm hello-node-container
hello-node-container

Desktop@DESKTOP-HTPKI6F MINGW64 ~/Documents/GitHub/Docker_Aplicacion_Node.js/hello-node (main)
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS
NAMES
```